tutorials.EU

Mastering Integration: API Calls, JSON Parsing, and Google Maps – Day 12 Android 14 Masterclass

Liza Cheremisina / 21. November 2023 / Android

Mastering Integration: API Calls, JSON Parsing, and Google Maps – Day 12 Android 14 Masterclass

Welcome to Day 12 of our Android 14 & Kotlin Masterclass. In this session, we explore integrating advanced functionalities into Android applications. We begin by guiding you through **setting up a Google Maps API key**, a pivotal step in integrating mapping capabilities. We then delve into the art of **making network requests using Retrofit**, an essential skill for modern app development that enables dynamic interactions with web

services. Additionally, we cover **JSON parsing**, a critical process for managing data in your apps. This comprehensive session is designed to elevate your Android development skills, empowering you to create interactive, data-driven, and user-friendly applications.

1. Setting up the API key for Google Maps

Setting up an API key for Google Maps for use in an Android application involves several steps, which include creating a project in the Google Cloud Platform, enabling the Google Maps API, and obtaining the API key. **Here's a step-by-step guide**:

https://developers.google.com/maps/documentation/android-sdk/get-api-key

Step 1: Google Cloud Project

1. **Go to the Google Cloud Console**: Navigate to Google Cloud Console.
2. **Create a New Project**: Click the project drop-down and then click on the "New Project" button at the top right. Enter a project name and choose a billing account as required.
3. **Select the Created Project**: After creating your project, select it from the project drop-down list at the top of the console page.

Step 2: Enable Google Maps API

1. **Open API Library**: From the dashboard, navigate to "API & Services" > "Library".
2. **Search for the API**: In the API Library, search for "Google Maps Android API" or "Maps SDK for Android."
3. **Enable the API**: Click on the appropriate API from the list and then click "Enable" to activate the API for your project.

Step 3: Obtain API Key

1. **Access API Credentials**: After enabling the API, you will be prompted to create credentials; click "Create credentials".

2. **Choose API Key**: In the “Create credentials” drop-down menu, select “API key”. Google will then generate a new API key for you.
3. **Restrict API Key (Optional but Recommended)**: After creating the key, you can restrict its use to your Android app to prevent unauthorized use. Click “Restrict key” and set the restrictions for the API key usage, like HTTP referrers (websites), IP addresses (server-side), or Android apps (Android-specific).

- For Android apps, you’ll need your package name and SHA-1 certificate fingerprint.
- Obtaining your SHA-1 certificate fingerprint for your Android app development can be done easily through the Gradle `signingReport` task, which is included in your Android project’s Gradle tasks, or via Terminal. Here’s how you can get it:

Using Android Studio:

1. **Open Your Project**: Start Android Studio and open your Android project.
2. **Gradle Panel**: Look for the Gradle panel on the right side of the Android Studio window.
3. **Run signingReport**:

- Expand the ‘Gradle’ panel on the right.
- Navigate through your project hierarchy: `YourProjectName -> Tasks -> android`.
- Double-click on the `signingReport` task. This will execute the `signingReport` task for your project.

4. **View Output**: The `signingReport` task will run and generate a report in the ‘Run’ panel at the bottom of Android Studio. In this report, you’ll see a section for each variant of your application (debug/release, etc.). Look for the ‘SHA1’ value within the report output.

Here’s an example of what the output might look like in the ‘Run’ output panel:

```
Variant: debugAndroidTest
Config: debug
Store: /home/user/.android/debug.keystore
Alias: AndroidDebugKey
MD5: A7:89:5A:...:E8:43
SHA1: BB:0D:AC:...:4E:F6
SHA-256: EF:49:30:...:C8:AA
Valid until: Friday, August 25, 2056
```

- **Copy the SHA-1 Value**: Find the line that starts with 'SHA1' and copy the fingerprint that follows it. This is the value you'll need when setting up your Google Maps API key or when configuring other services that require your app's SHA-1 certificate fingerprint.

Using the Terminal:

If you're comfortable using the command line, you can also run the **`signingReport`** task from your project's root directory:

1. Open a terminal window.
2. Navigate to your project's root directory using the **`cd`** command.
3. Once you're in your project's root directory, execute the following command:

```
./gradlew signingReport
```

(Use **`gradlew.bat signingReport`** if you're on Windows.)

The **`signingReport`** output will appear in your terminal window, where you can find the SHA-1 value as described above.

Remember, the SHA-1 fingerprint from the **`signingReport`** will be for the keystore that's currently configured in your **`build.gradle`** file for the app module. If you're using the debug keystore, it will show the SHA-1 for the debug mode.

If you need the SHA-1 for the release keystore, make sure you've configured the release keystore details in your **`build.gradle`** file and are building for release.

Step 4: Use the API Key in Your Android App

1. **Add the Key to Your Project**: Once you have the API key, add it to your Android project. Open your project in Android Studio.
2. **Update the Manifest**: Add a meta-data element to your application's AndroidManifest.xml file with the name **`com.google.android.geo.API_KEY`** and a value of your API key:

```
<manifest>
    <application>
        <!-- Add the following meta-data element inside the <application>
node -->
        <meta-data
            android:name="com.google.android.geo.API_KEY"
            android:value="YOUR_API_KEY_HERE"/>
        <!-- Make sure the rest of your application elements are below -->
    </application>
</manifest>
```

3. **Use the Maps SDK**: Now that your API key is set up, you can use the Google Maps SDK as per its documentation to add maps to your app.

4. **Testing**: Run your application and test to make sure the map displays correctly.

Remember to keep your API key secret to prevent misuse, and monitor your usage to stay within the free usage limits or your preferred budget if you have billing set up.

Always refer to the latest Google documentation and pricing structures, as these can change over time.

2. Restricting your Google Maps API key

Restricting your Google Maps API key is an **important security measure to prevent unauthorized use of your key**. Here's how you can restrict your API key to only work with your Android app:

1. Find your API Key in Google Cloud Console:

After you have created your API key by following the steps to set it up, you can go back to the Google Cloud Console to manage it.

2. Navigate to API Credentials:

- Go to the APIs & Services > Credentials page on the Google Cloud Console.
- Find the API key that you want to restrict. Click on the name of the key to go to the settings page for that key.

3. Restrict your API Key:

On the API key settings page, you'll find several options for restricting your API key, such as:

- **Application restrictions**: Choose "Android apps" and add your Android app's package name and SHA-1 certificate fingerprint. This restricts the use of the key to only your Android app. You can find your package name in your app's `build.gradle` file under the `applicationId` field. The SHA-1 certificate fingerprint can be obtained as described in the previous messages using the `signingReport` Gradle task in Android Studio.
- **API restrictions**: Click "Restrict key" and select the specific Google APIs you want this key to access. For Google Maps on Android, you should restrict it to the "Maps SDK for

Android". This means your API key can only be used to access the Maps SDK for Android and no other APIs.

Here's a more detailed walkthrough for each restriction type:

Application Restrictions:

1. Under "Application restrictions," select "Android apps."
2. In the "Package name" field, enter your app's package name exactly as it appears in your `build.gradle` file or your app's manifest.
3. In the "SHA-1 certificate fingerprint" field, enter the SHA-1 fingerprint you obtained using the `signingReport` task.
4. Click "Add an item" if you want to add another app (for instance, if you have separate debug and release keys).

API Restrictions:

1. Under "API restrictions," you can choose to "Don't restrict key" (not recommended) or "Restrict key" (recommended for enhanced security).
2. If you choose "Restrict key," a list of APIs will appear. Select "Maps SDK for Android" from the list of available APIs.
3. If you use any other related services, like Places API, you should select those as well.

4. Save your settings:

After you've set the restrictions, be sure to click "Save" at the bottom of the page to apply the restrictions to your API key.

Best Practices for API Key Security:

- **Do not embed API keys directly in code:** Instead, use a secure method to load the key into your app. You can use mechanisms like Android's secure properties or encrypted files.

- **Do not store API keys in your app's version control system**: If you're using Git or another version control system, exclude files that contain API keys using `.gitignore` or equivalent mechanisms in other systems.
- **Review API key activity regularly**: Check the usage reports in the Google Cloud Console to monitor for unexpected use of your API key.

By following these steps and best practices, you can help ensure that your API key is only used as intended and reduce the risk of unauthorized access.

3. What is LatLng object?

Theory:

The term "LatLng" is short for "Latitude and Longitude." Latitude and Longitude are two coordinates that uniquely identify any location on the Earth's surface.

Latitude is a measure of how far north or south a location is from the Equator, and Longitude is a measure of how far east or west a location is from the prime meridian that runs through Greenwich, England.

A **LatLng object**, in many mapping APIs including Google Maps, is simply a **way of storing these two coordinates into a single entity/object** so that the map knows where to display a marker, draw a path, or set the map view.

Syntax:

In Android using the Google Maps API, the `LatLng` class is used to create such an object.

It takes : latitude and longitude, both of which are double values (where `double` is a type of number that includes digits after the decimal point to allow for precision).

```
LatLng(latitude: Double, longitude: Double)
```

Code Examples:

Here are some code examples that show how to use the LatLng object in different scenarios:

Creating a LatLng Object:

```
// Create a LatLng object for the city of New York, where:
// - The latitude is 40.7128 degrees North
// - The longitude is -74.0060 degrees West
val newYorkLatLng = LatLng(40.7128, -74.0060)
```

Using LatLng with a Map to Add a Marker:

```
// Assuming you have a GoogleMap object already set up as 'map'
val sydneyLatLng = LatLng(-33.8688, 151.2093) // Sydney, Australia coordinates
map.addMarker(MarkerOptions().position(sydneyLatLng).title("Marker in Sydney"))
```

Centering the Map on a Location:

```
// Set the map to center on the LatLng for Sydney
map.moveCamera(CameraUpdateFactory.newLatLng(sydneyLatLng))
```

Zooming to a Location:

```
// Center the map on Sydney and zoom in closer
val zoomLevel = 10.0f // Float value where 1.0f is world view and 21f is
street view
map.moveCamera(CameraUpdateFactory.newLatLngZoom(sydneyLatLng, zoomLevel))
```

Drawing a Path Between Two Locations:

To draw a path or a line between two locations on a map, you can use a Polyline object that connects multiple LatLng points.

```
// Create LatLng objects for two locations
val origin = LatLng(37.7749, -122.4194) // San Francisco, CA
val destination = LatLng(34.0522, -118.2437) // Los Angeles, CA

// Add polyline to the map
val polylineOptions = PolylineOptions().add(origin, destination)
map.addPolyline(polylineOptions)
```

To use these features, make sure you've set up Google Maps correctly in your Android project, including getting an API key and adding the necessary permissions and map setup code in your app.

4. Making Network Requests with Retrofit: @Query

`@Query` is an annotation used in Android development when making network requests, particularly with Retrofit, which is a type-safe HTTP client for Android and Java.

Theory:

`@Query` is used to annotate parameters in a method declaration for an interface in Retrofit. Each instance of `@Query` represents a query key in the URL request where its value will be dynamically provided by the method parameter's value.

When the HTTP request is made, Retrofit takes each parameter annotated with `@Query`, encodes it as a string if necessary, and appends these as query parameters to the request URL.

For example, if you are trying to access a web service that provides geographical data and you need to pass the latitude and longitude coordinates as query parameters, you use `@Query` to append them to the URL.

Syntax:

The syntax for using `@Query` in a Retrofit interface method is as follows:

```
@Query("query_key") parameterVariable: ParameterType
```

Here, `"query_key"` is the exact key that the API expects for the query parameter in the URL. `parameterVariable` is the name of the parameter you use in your method, and `ParameterType` is the type of data that variable holds, like `String`, `Int`, `Double`, etc.

Code Examples:

Let's assume you have a Retrofit interface that fetches some location-based information using latitude and longitude coordinates.

The web service expects the parameters `latlng` for coordinates and `key` for the API key. Your Retrofit interface method could look something like this:

```
interface LocationService {
    @GET("location/info")
```

```
    suspend fun getLocationInfo(
        @Query("latlng") latlng: String, // `latlng` will be used as a
query parameter in the URL
        @Query("key") apiKey: String // `key` will be used as another query
parameter in the URL
    ): Response<LocationData> // Assume LocationData is a data class that
models the JSON response.
}
```

Using the `@Query` Annotated Method:

```
// Create an instance of Retrofit
val retrofit: Retrofit = // ... initialize Retrofit instance

// Create an implementation of the API endpoints defined by the interface
val locationService: LocationService =
retrofit.create(LocationService::class.java)

// Call the method with the appropriate parameters
val response: Response<LocationData> = locationService.getLocationInfo(
    "40.7128,-74.0060", // Pass coordinates as a string separated by a
comma
    "your_api_key_here" // Your API key
)
```

When this method is called, Retrofit generates an HTTP GET request to **`location/info?latlng=40.7128,-74.0060&key=your_api_key_here`**.

The **`@Query`** annotation **makes it easy to add query parameters to a URL without manually constructing the query string.**

Retrofit takes care of encoding the parameters and attaching them to the URL, which prevents common mistakes like typos in parameter names or incorrect URL formatting.

5. Retrofit Converter Factory

Let's look closer at this line of code: **.addConverterFactory(GsonConverterFactory.create())**

Retrofit Converter Factory:

- **Converter Factories**: Retrofit uses the concept of converter factories to encode and decode data. You add a converter factory to Retrofit's builder so that it knows how to handle the data according to a specific format or data protocol.

Gson Converter Factory:

- **Gson**: Gson is a Java library that can be used to convert Java Objects into their JSON representation and vice versa. It can work with arbitrary Java objects including pre-existing objects that you do not have source-code of.
- **GsonConverterFactory**: This is a converter factory class specifically for Gson. When you add this factory to Retrofit, it allows Retrofit to use Gson for JSON parsing. This means any data received from a web service will be parsed from JSON to Java objects, and any Java objects sent to the web service will be serialized to JSON format.

The Method `.create()`:

- **create()**: This method creates a new instance of the GsonConverterFactory. It prepares the factory for use with Retrofit.

Putting it all together in Retrofit:

When setting up a Retrofit instance, you configure it using a **`Retrofit.Builder`** object, which provides a fluent interface to add various settings. The **`.addConverterFactory()`** method is a part of this builder pattern and is used to attach the Gson converter factory to Retrofit's configuration.

```
val retrofit = Retrofit.Builder()
    .baseUrl("<https://api.example.com>") // Sets the API base URL
    .addConverterFactory(GsonConverterFactory.create()) // Tells Retrofit
to use Gson for JSON parsing
    // ... (any other configuration like call adapters etc.)
    .build() // Creates the Retrofit instance with the provided settings
```

With GsonConverterFactory attached, **Retrofit can automatically handle the conversion of JSON data to/from Java objects** based on the Gson rules and the annotations you've provided in your data model classes.

Here's what happens when you make a network request with Retrofit configured with Gson:

1. **Serialization**: When sending data to a server, Retrofit serializes the request body objects into JSON using Gson.
2. **Deserialization**: When receiving a response from a server, Retrofit parses the JSON response body into Java objects using Gson.

In essence, `.addConverterFactory(GsonConverterFactory.create())` is a critical configuration line that makes JSON handling with Retrofit simple and seamless, freeing the developer from manual parsing and error-prone boilerplate code.

6. Write a Log Message to LogCat within Android Studio

```
Log.d(tag: "res1”, msg: ${e.cause} ${e.message}")
```

Here's a breakdown of the components and actions in this line:

- `Log`: This is the class provided by the Android SDK to send log messages.
- `.d`: This stands for “debug”. The `d` method is used to write debug level log messages. Debug logs are useful for printing out information that might be needed for diagnosing issues during development but typically not useful in a production environment.
- `tag`: This is a string that is used to identify the source of a log message. Tags are typically used to filter LogCat messages to find those that are relevant to a particular issue or part of your app.
- `"res1"`: In your example, “res1” is the string literal provided as the tag for this log message. When you filter LogCat by this tag, you’ll see all messages logged with it.
- `msg`: This is the message that you want to log. Log messages can be a combination of static text and variable values.
- `${e.cause}` and `${e.message}`: This syntax is used to insert the value of an expression within a string in Kotlin. `e` is a reference to an exception object, `e.cause` is the throwable that caused the exception (the root cause), and `e.message` provides the detailed message string of the exception.
- The overall `msg` in the example combines the cause and the message of an exception into a single string, separated by spaces.

What you’re doing with this line of code is essentially **printing out debugging information related to an exception that was caught in your app**. When the code runs, and an exception `e` occurs, this line will create a log entry that contains the root cause and the message of that exception.

This information can be very useful when trying to understand why an error occurred; knowing both the message and the cause can provide context that’s needed for troubleshooting.

Using LogCat for such logging is a common practice in Android development, as it allows you to record detailed information about the behavior of your application, which you can then view in the LogCat pane within Android Studio.

What is a LogCat?

LogCat, short for “Log Catalog,” is a command-line tool integrated into Android Studio that collects and displays a log of system messages, including stack traces when the device throws errors, and messages that you have written from your app with the Log class.

Here’s a more detailed look at what LogCat does and how developers typically use it:

Functionality:

- **Capturing Logs**: LogCat captures all log messages from applications running on an Android device or emulator, along with messages from the Android system itself. It collects various types of logs such as verbose, debug, info, warning, error, and assert.
- **Filtering Logs**: Developers can filter the displayed log messages by different criteria, including log level, tag, process ID, thread ID, or simply by searching for specific content within the messages.
- **Real-time Monitoring**: LogCat updates in real-time as messages are written to the log, allowing developers to monitor the behavior of their applications as they are running.
- **Debugging Aid**: By logging relevant information at strategic points in an application’s code, developers can use LogCat to help debug applications and understand the flow of execution and data.

7. The `.firstOrNull()` Method in Kotlin

The `.firstOrNull()` method in Kotlin is a standard library extension function for iterables, such as lists or any other collection that can be iterated over.

This method returns the first element from a collection that matches a given predicate or returns `null` if no such element is found or if the collection is empty.

Functionality:

- **Returning the First Element**: If you call `.firstOrNull()` without a predicate, it will return the first element of the collection if the collection is not empty. If the collection is

empty, it returns `null`.

- **Using a Predicate**: You can also pass a lambda expression to `.firstOrNull()` that defines a condition (predicate). The method then returns the first element that matches this predicate. If no elements match the predicate, or the collection is empty, `null` is returned.

Syntax:

The syntax for `.firstOrNull()` without a predicate is:

```
collection.firstOrNull()
```

And with a predicate, it is:

```
collection.firstOrNull { it.someProperty == someValue }
```

Here, `it` refers to the current element in the iteration, `someProperty` is a property or value you want to check, and `someValue` is the value you're looking for.

Examples:

Here's an example without a predicate:

```
val numbers = listOf(1, 2, 3, 4)
val firstNumber = numbers.firstOrNull() // This will return 1
```

And here's an example with a predicate:

```
val people = listOf("Alice", "Bob", "Carol")
val firstPersonWithA = people.firstOrNull { it.startsWith("A") } // This
will return "Alice"
val firstPersonWithZ = people.firstOrNull { it.startsWith("Z") } // This
will return null, as no names start with "Z"
```

Common Use Case:

The `.firstOrNull()` method is especially useful when you're working with collections and you expect that there might be no element matching your criteria, or when the collection might be empty. Using `.firstOrNull()` helps avoid throwing a `NoSuchElementException`, which would happen if you used `first()` in such situations.

In summary, `.firstOrNull()` is a concise and safe way to retrieve potentially non-existing elements from a collection based on custom conditions, while avoiding exceptions associated with unfound elements.

8. Understanding API Calls

API calls are essentially the way in which software applications communicate with each other. "API" stands for Application Programming Interface, which is a set of rules, protocols, and tools for building software and applications.

When an application makes an API call, it sends a request to a server to do something – like retrieve data or execute an operation – and then waits for the server's response.

Here's a breakdown of the concept:

1. Request: The initiating action taken by the client software. It's composed of:

- **Endpoint**: The URL to which the request is sent. It typically denotes the specific data or function being requested.
- **Method**: The type of action to be performed. Common HTTP methods include GET (retrieve data), POST (submit data), PUT (update data), DELETE (remove data), etc.
- **Headers**: Provide metadata about the request, such as the content type, authentication information, etc.
- **Body**: Contains data sent by the client to the server (not used with GET methods), often in the form of JSON or XML.

2. **Response**: The answer returned from the server. It usually contains:

- **Status Code**: Indicates success or failure (like 200 for success, 404 for not found, etc.).
- **Headers**: Similar to request headers, they provide metadata about the response.
- **Body**: The data sent back by the server to the client, typically in a structured format like JSON or XML.

How API Calls Work

Imagine you're using a weather application on your smartphone. When you request the forecast:

1. The app makes an API call to a weather service's server.
2. The app specifies the endpoint (e.g., a URL for the weather forecast).
3. The app includes parameters such as your location and the format you want the data in.
4. The server processes the request and sends back the requested weather data.
5. The app receives this data and displays the forecast to you.

Why API Calls are Important

API calls are critical in the modern software ecosystem because they allow for the integration of different services and platforms. They enable apps to access functionalities provided by external services without having to recreate those functionalities themselves. For instance, a travel booking application might use API calls to access current flight data from various airlines.

REST API Calls

Most API calls on the web are RESTful. REST (Representational State Transfer) is an architectural style that uses standard HTTP methods and stateless communication for interacting with web services. RESTful APIs are designed to be easy to understand and use, and they are the backbone of many services on the internet.

In summary, API calls are the mechanism by which applications outsource tasks to servers over a network, allowing them to leverage external data and functionality. They are an essential component of software development that enables interoperability between different systems and services.

Conclusion: Mastering Integration: API Calls, JSON Parsing, and Google Maps – Day 12 Android 14 Masterclass

As we conclude Day 12 of our Android 14 & Kotlin Masterclass, we've ventured through the complexities and nuances of integrating APIs, parsing JSON, and utilizing Google Maps in Android applications. The insights and skills gained today are invaluable for any developer looking to enhance their app's interactivity and functionality. By understanding the intricacies of Retrofit for network requests and the importance of securing API keys, we are better equipped to build applications that are not only feature-rich but also robust and secure.

If you want to skyrocket your Android career, check out our The Complete Android 14 & Kotlin Development Masterclass. Learn Android 14 App Development From Beginner to Advanced Developer.

Master Jetpack Compose to harness the cutting-edge of Android development, **gain proficiency in XML** — an essential skill for numerous development roles, and **acquire practical expertise by constructing real-world applications**.

Check out Day 10 of this course here.

Check out Day 11 of this course here.

Check out Day 13 of this course here.

Related Posts:

1. Navigating Libraries, APIs, and Remote Content – Day 9 Android 14 Masterclass
2. Exploring Basic Kotlin Syntax and Structure – Day 2 Android 14 Masterclass
3. Basic Kotlin Syntax: Functions, Objects and Classes – Day 3 Android 14 Masterclass
4. Mastering Screen Navigation with Kotlin – Day 10 Android 14 Masterclass

Tags: .FIRSTORNULL() ANDROID 14 API CALLS API KEY API RESTRICTIONS CONVERTER FACTORY FUNCTION GOOGLE CLOUD PROJECT GOOGLE MAPS API JSON KOTLIN KOTLIN SYNTAX LATLNG LATLNG OBJECT LOG MESSAGE LOGCAT METHOD NETWORK REQUESTS PARSING QUERY REST API RETROFIT SDK SHA-1 SIGNINGREPORT